Syllabus

Chapter 8 : Hibernate Introduction (4hrs)

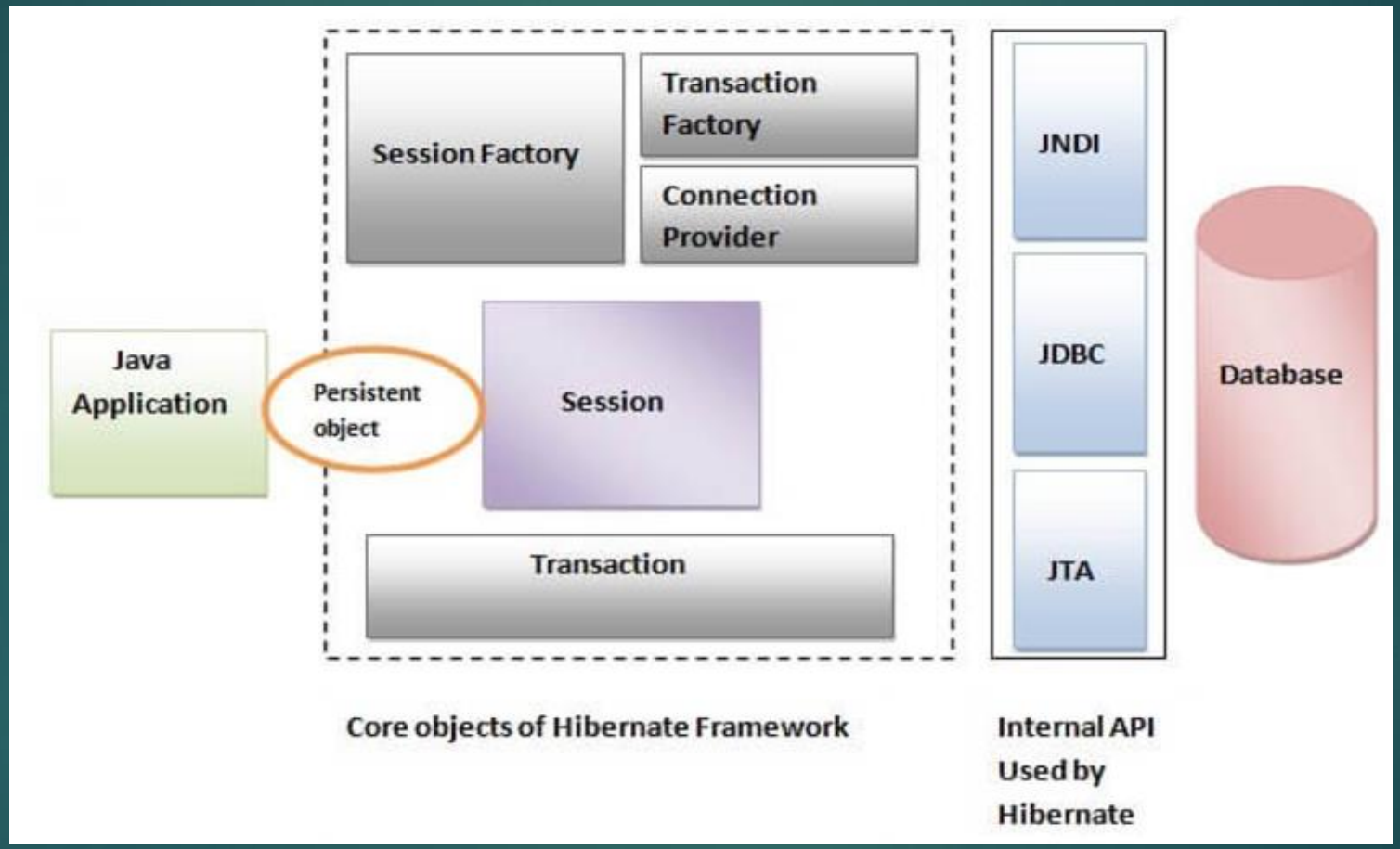
1. Introduction to Hibernate 3.0
2. Hibernate Architecture
3. First Hibernate Application.
4. First Hibernate Application contd.

Hibernate

Hibernate Framework

- Hibernate is a Java framework that simplifies the development of Java application to interact with the database
- It is an open source, lightweight, ORM (Object Relational Mapping) tool.
- Hibernate implements the specifications of JPA (Java Persistence API) for data persistence.
- ORM tool
  - An ORM tool simplifies the data creation, data manipulation and data access.
  - It is a programming technique that maps the object to the data stored in the database.

Architecture



Architecture

- Hibernate has a layered architecture which helps the user to operate without having to know the underlying APIs.
- Hibernate uses the database and configuration data to provide persistence services (and persistent objects) to the application.
- The following section gives a brief description of each of the class objects involved in Hibernate Application Architecture.

Architecture

- **SessionFactory**
  - The SessionFactory is a factory of the session and client of ConnectionProvider.
  - It holds a second-level cache (optional) of data.
  - The org.hibernate.SessionFactory interface provides a factory method to get the object of Session.
- **Session**
  - The session object provides an interface between the application and data stored in the database.
  - It is a short-lived object and wraps the JDBC connection.
  - It is factory of Transaction, Query and Criteria.
  - It holds a first-level cache (mandatory) of data.
  - The org.hibernate.Session interface provides methods to insert, update and delete the object.
  - It also provides factory methods for Transactions, Query, and Criteria.

Architecture

- **Transaction**
  - The transaction object specifies the atomic unit of work.
  - It is optional.
  - The org.hibernate.Transaction interface provides methods for transaction management.

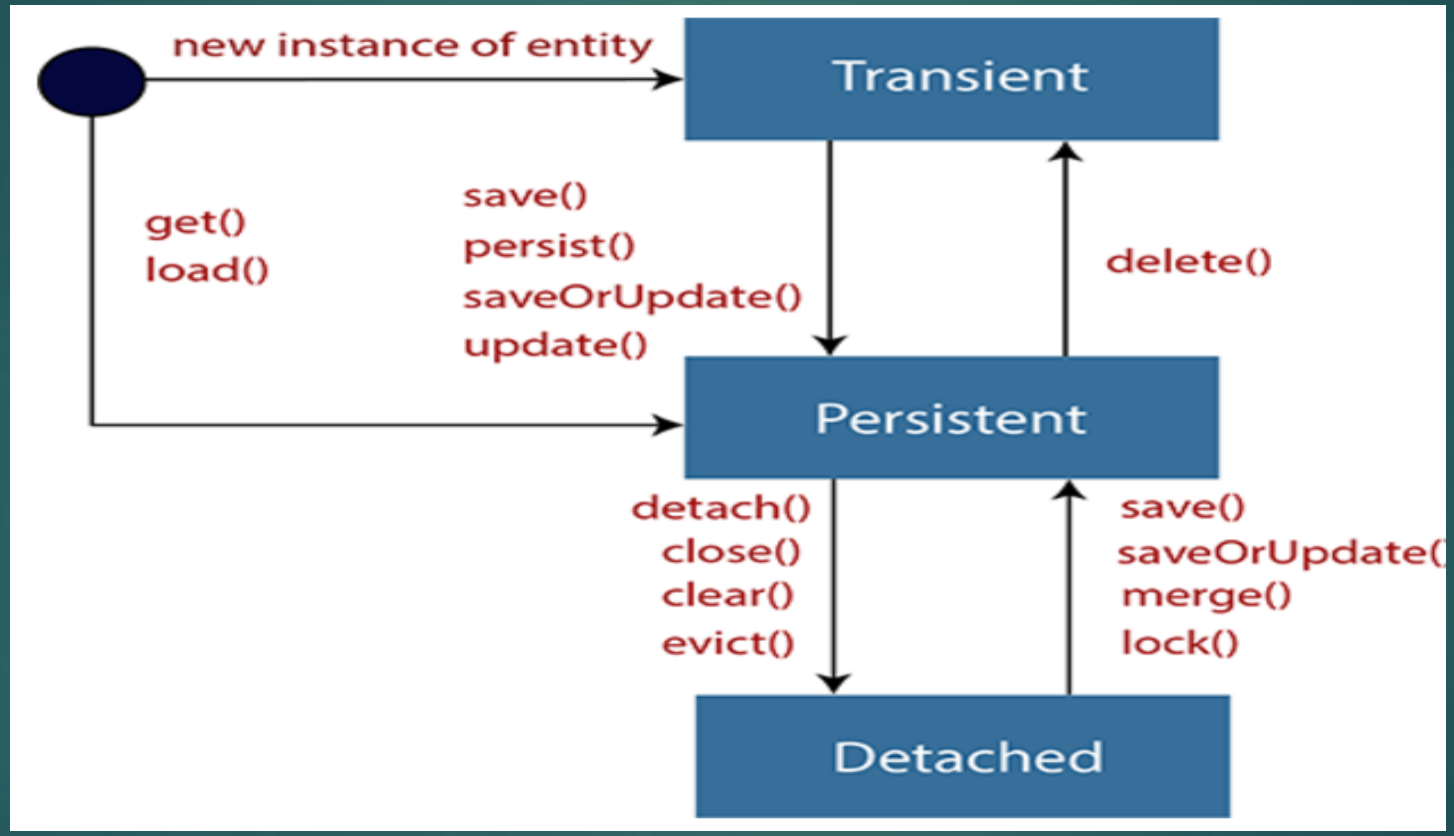
- **ConnectionProvider**
  - It is a factory of JDBC connections.
  - It abstracts the application from DriverManager or DataSource.
  - It is optional.

- **TransactionFactory**
  - It is a factory of Transaction.
  - It is optional..

Hibernate Life Cycle

- In Hibernate, either we create an object of an entity and save it into the database,
- or we fetch the data of an entity from the database.
- Here, each entity is associated with the lifecycle.
- The entity object passes through the different stages of the lifecycle.
- The Hibernate lifecycle contains the following states: -
  - Transient state
  - Persistent state
  - Detached state

Hibernate Life Cycle



Hibernate Life Cycle

- **Transient state**
  - The transient state is the initial state of an object.
  - Once we create an instance of POJO class, then the object entered in the transient state.
  - Here, an object is not associated with the Session. So, the transient state is not related to any database.
  - Hence, modifications in the data don't affect any changes in the database.
  - The transient objects exist in the heap memory. They are independent of Hibernate.
- Example :

```
Employee e=new Employee();     //Here, object enters in the transient state.
e.setId(101);
e.setFirstName("Gaurav");
e.setLastName("Chawla");
```

Hibernate Life Cycle

- **Persistent state**
  - As soon as the object is associated with the Session, it enters the persistent state.
  - Hence, we can say that an object is in the persistence state when we save or persist it.
  - Here, each object represents the row of the database table.
  - So, modifications in the data make changes in the database.
- Example
  - We can use any of the following methods for the persistent state
    - session.save(e);
    - session.persist(e);
    - session.update(e);
    - session.saveOrUpdate(e);
    - session.lock(e);
    - session.merge(e);

Hibernate Life Cycle

- **Detached State**
  - Once we either close the session or clear its cache, then the object enters into the detached state.
  - As an object is no longer associated with the Session, modifications in the data don't affect any changes in the database.
  - However, the detached object still has a representation in the database.
  - If we want to persist with the changes made to a detached object, the application must be reattached to a valid Hibernate session.
  - To associate the detached object with the new hibernate session, use any of these methods – load(), merge(), refresh(), update() or save() on a new session with the reference of the detached object.
- Example : We can use any of the following methods for the detached state

Hibernate Life Cycle

- Example : We can use any of the following methods for the detached state
  - session.close();
  - session.clear();
  - session.detach(e);
  - session.evict(e);

First Hibernate Application

- Folder Structure

```
MyFirstHibernateApp/
  ├── src/main/java/
  │     ├── com/example/
  │     │      ├── entity/Student.java
  │     │      └── MainApp.java
  │
  ├── src/main/resources/
  │     └── hibernate.cfg.xml
  │
  └── pom.xml
```

pom.xml

- Add Hibernate + MySQL + Logging dependencies.

```
<dependencies>
    <!-- Hibernate Core -->
    <dependency>
        <groupId>org.hibernate.orm</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>6.4.4.Final</version>
    </dependency>
    <!--  MySQL JDBC Driver -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <version>9.0.0</version>
    </dependency>
```

pom.xml

- Add Hibernate + MySQL + Logging dependencies.

```
    <!-- Logging (for console output) -->
    <dependency>
        <groupId>org.jboss.logging</groupId>
        <artifactId>jboss-logging</artifactId>
        <version>3.5.3.Final</version>
    </dependency>
</dependencies>
```

Create Database In MySQL:

- CREATE DATABASE hibernate_demo;
- USE hibernate_demo;

hibernate.cfg.xml

- Configuration file inside src/main/resources/

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 6.0//EN"
        "http://hibernate.org/dtd/hibernate-configuration-6.0.dtd">

<hibernate-configuration>
    <session-factory>
        <!-- Database connection settings -->
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/hibernate_demo</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">your_password</property>
```

hibernate.cfg.xml

```
    <!-- JDBC connection pool (use built-in) -->
    <property name="connection.pool_size">1</property>

    <!-- SQL dialect -->
    <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>

    <!-- Show generated SQL in console -->
    <property name="show_sql">true</property>

    <!-- Automatically create/update tables -->
    <property name="hbm2ddl.auto">update</property>

    <!-- Mapping class -->
    <mapping class="com.example.entity.Student"/>

  </session-factory>
</hibernate-configuration>
```

Student.java (Entity Class)

- Inside src/main/java/com/example/entity/

```
package com.example.entity;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;


@Entity
@Table(name = "student")
public class Student {
```

Student.java (Entity Class)

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;
private String name;
private String email;

public Student() {}

public Student(String name, String email) {
    this.name = name;
    this.email = email;
}
```

Student.java (Entity Class)

```
    public int getId() { return id; }
     public String getName() { return name; }
     public void setName(String name) { this.name = name; }
     public String getEmail() { return email; }
     public void setEmail(String email) { this.email = email; }


     @Override
     public String toString() {
         return "Student [id=" + id + ", name=" + name + ", email=" + email + "]";
     }
}
```

MainApp.java

- Inside src/main/java/com/example/

```
package com.example;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import com.example.entity.Student;

public class MainApp {
    public static void main(String[] args) {
        // Create SessionFactory
        SessionFactory factory = new Configuration()
                .configure("hibernate.cfg.xml")
                .addAnnotatedClass(Student.class)
                .buildSessionFactory();
```

MainApp.java

```
        // Create Session
        Session session = factory.getCurrentSession();

        try {
            // Create a new Student object
            System.out.println("Creating new student object...");
            Student student = new Student("John Doe", "john@example.com");

            // Start a transaction
            session.beginTransaction();

            // Save the student object
            System.out.println("Saving the student...");
            session.persist(student);

            // Commit transaction
            session.getTransaction().commit();
            System.out.println("Done! Saved: " + student);
        } finally {
            factory.close();
        }
    }
```

How it works

- Hibernate loads configuration from hibernate.cfg.xml
- SessionFactory connects to MySQL
- Entity class Student maps to student table
- Session begins transaction and inserts data
- Hibernate auto-generates SQL: insert into student (name, email) values (?, ?)
- Data saved successfully

Sample Questions

1. With a neat diagram , explain Hibernate Architecture.
2. With a neat diagram , explain entity life cycle in Hibernate.
3. Write a simple hibernate program to save students details like usn , name and email in MySql database.
4. Write a simple hibernate program to save user details like firstname , lastname ,phone and email in MySQL database.
5. Write a simple hibernate program to save employee details like empId , empName, EmpSalary in MySQL database.